

Intelligent SOC-Aware Bay Allocation for Multi-Bay EV Charging Stations

A systems analysis of dynamic, capacity-matched, fairness-aware charging bay scheduling

Prepared by: Naveen | Project scoping note

1. The Core Idea

In a multi-bay EV charging station (e.g. 6 bays), cars are conventionally served first-come-first-served, with no regard to how much charge each car actually needs or how soon each bay will free up. The proposed concept replaces this with **dynamic, SOC-aware bay allocation**: the system continuously tracks the remaining charge time of every occupied bay and, whenever a new car joins the queue, matches it to the bay that minimizes overall waiting time across the system — a car needing only a small top-up should be routed to the bay finishing soonest, rather than sitting behind a car that needs a near-full charge.

This is structurally an **online scheduling / constrained assignment problem**, related to Shortest Remaining Processing Time (SRPT) scheduling in operating systems and to job-shop scheduling with machine-eligibility constraints.

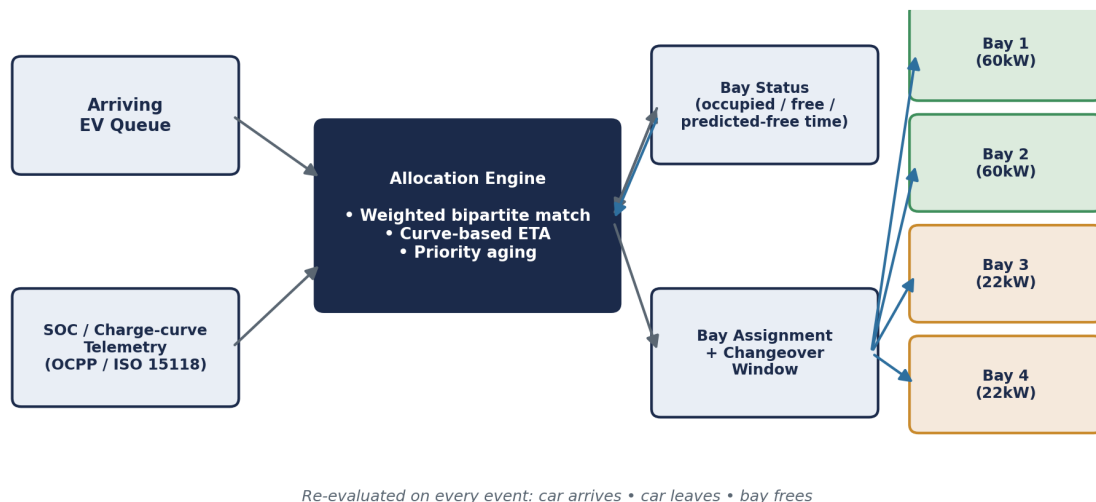


Figure 1. High-level data flow: telemetry and queue state feed a central allocation engine, which re-solves the assignment on every event and pushes bay assignments back out.

2. What Already Exists

The general problem of EV charging scheduling is an active research area, but true dynamic bay-to-car matching by remaining charge time is **not commonly deployed commercially**. Most public stations

(Tesla Superchargers, ChargePoint, etc.) still operate on simple first-come-first-served or basic queue notification systems. What does exist:

Deployed / patented systems

- **Queue prioritization patents** (US9142978B2, US20140125279A1) — reorder who charges next based on operator-defined rules, notify the user at the top of the queue, and enforce a changeover window to occupy the bay before forfeiting the slot. This addresses *ordering*, not *bay-matching*.
- **EVCS queue management patents** (US20140125485A1, WO2014074425A2) — solve the "car sits at a free bay long after it's full" problem via time-limited slot enforcement.
- **App-based queueing proposals** — real-time queue position, license-plate verification, and a short grace window for the next driver to occupy a freed bay, functioning as a standalone layer independent of the charger hardware.

Research literature

- **Greedy / heuristic SOC-based scheduling** — algorithms that assign charging slots iteratively based on current battery level, predicted demand, and grid cost, favoring simplicity over global optimality.
- **Online scheduling with competitive-ratio guarantees** (e.g. FOCS-type algorithms) — allocate limited charging resource across EVs arriving dynamically, with proven worst-case bounds versus the optimal offline schedule.
- **Mechanism design / auction-based allocation** — treats bay assignment as a market-clearing problem where EVs report preferences and the system periodically clears the queue.
- **Occupancy-aware routing** — long-distance EV routing research that factors real-time charger occupancy signals into path planning, though this targets station selection, not intra-station bay matching.

Conclusion: the pieces (SOC-based prioritization, changeover handling, online scheduling theory) exist separately in patents and papers, but a combined system that does capacity-matched, fairness-aware, curve-accurate bay assignment within a single station appears to be a genuine gap — realistic as a student project / IEEE conference paper direction.

3. Issues With the Naive Version

3.1 Non-linear charging curve

Li-ion cells charge in two phases: Constant Current (CC) up to roughly 70–80% SOC, then Constant Voltage (CV) with a hard taper. The last 20% often takes as long as the first 70%. Allocating bays using raw percentage-remaining as a proxy for time-remaining will systematically misjudge which bay frees up soonest, especially for any car above ~75% SOC.

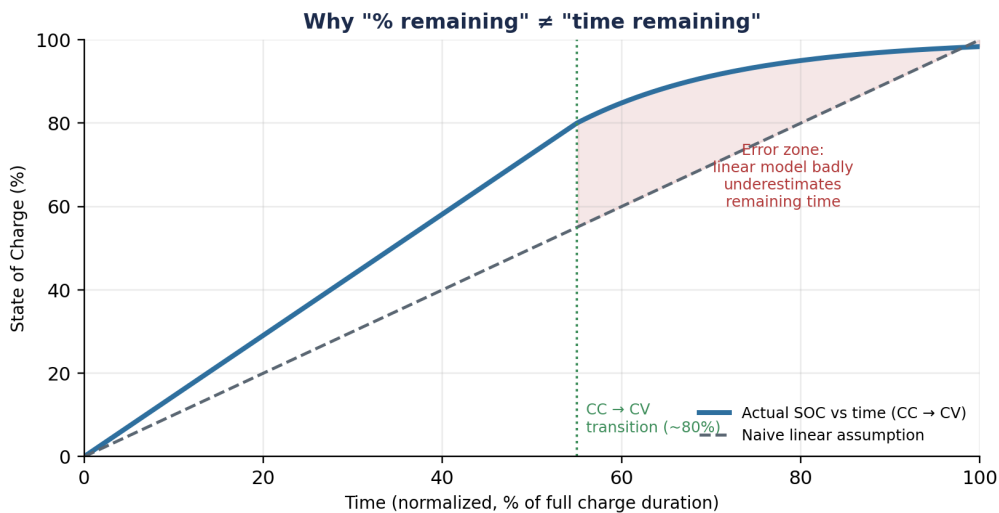


Figure 2. Real CC/CV charging behavior vs. the naive linear-percentage assumption. The gap widens sharply above ~80% SOC.

3.2 Battery capacity differs across vehicles

10% remaining on a 40 kWh car is a very different energy quantity than 10% on a 100 kWh car. The system needs kWh-remaining ($\text{SOC} \times \text{capacity}$), not raw percentage, to produce comparable time estimates across different vehicle models.

3.3 Bay power rating vs. vehicle acceptance rate

Not all bays are rated equally, and not all vehicles can accept a bay's full power. Effective charging power for any pairing is $\min(\text{bay_kW}, \text{vehicle_max_kW})$. A high-capacity vehicle assigned to a low-rated bay will charge far slower than the raw bay-vs-bay comparison suggests, and a small car on a high-power bay wastes that bay's capability.

3.4 Starvation under greedy short-job-first allocation

If the system always routes the smallest-need car to the soonest-free bay (pure Shortest-Remaining-Time-First), a car needing an 80% charge can be repeatedly overtaken by a continuous stream of smaller top-up requests. This is a known pathology in SRPT-style scheduling: fine under light queue load, but severe once the queue is long and arrivals are frequent — exactly the condition flagged as the real risk.

3.5 Telemetry access

Accurate real-time SOC and charge-curve data isn't something the station can observe externally by default. It requires vehicle-side integration (ISO 15118) and station-side protocol support (OCPP), or fallback to user self-reported SOC, which is unverifiable and gameable.

3.6 Changeover overhead is not zero

Physically swapping a car out of a bay and the next one in takes real time (walking to the car, unplugging, repositioning, plugging in). Treating bay transitions as instantaneous will make the schedule systematically over-optimistic.

3.7 Prediction uncertainty and no-shows

Users unplug early, leave the bay occupied after finishing, or don't show up when notified. Any schedule built on point-estimate predictions needs slack and re-evaluation triggers, not a one-shot plan.

4. Proposed System Design

4.1 Reframing as constrained optimal assignment

Every time a bay frees up or a new car joins the queue, re-solve a **weighted bipartite matching** between waiting cars and available/soon-to-be-available bays (Hungarian algorithm or a simpler greedy approximation for embedded-scale queues). Bay incompatibility is treated as a heavy cost penalty rather than a hard exclusion, so an oversized vehicle can still use a lower-rated bay when it's genuinely the best available option, instead of being blocked outright.

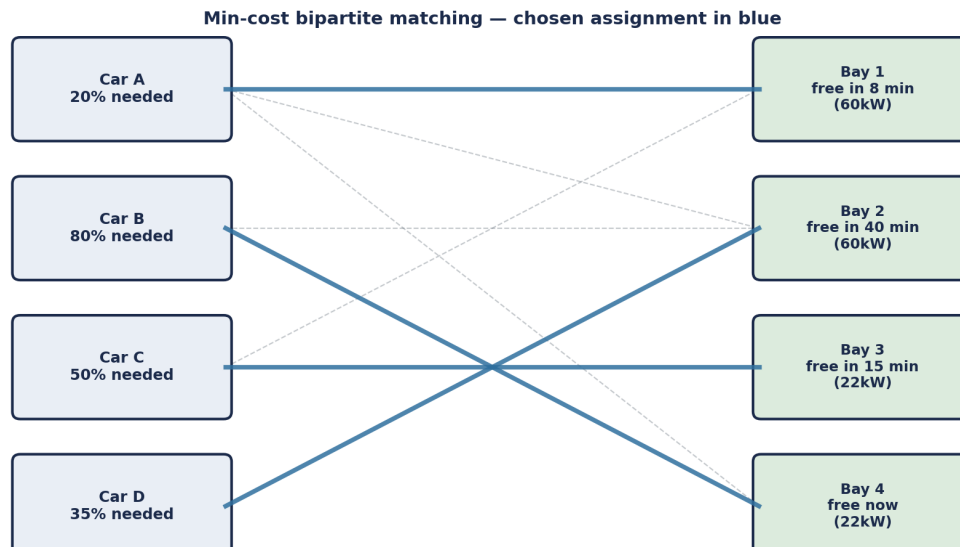


Figure 3. Min-cost bipartite matching example — each waiting car is paired with the bay that minimizes its effective completion time, not simply the nearest or soonest-free one.

4.2 Accurate time estimation

$\text{time_to_target}(\text{car}, \text{bay}) = f(\text{kwh_needed}, \text{charge_curve_model}, \min(\text{bay_kw}, \text{vehicle_max_kw}))$

using a piecewise CC/CV curve rather than a linear percentage assumption.

4.3 Fairness via priority aging

To prevent the starvation case, blend charge-time-needed with elapsed wait time, borrowed from the same "aging" principle used in OS process schedulers (e.g. Linux CFS virtual-runtime fairness):

$\text{priority}(\text{car}) = \text{charge_time_needed} / (1 + k \times \text{wait_time_elapsed})$

Lower score is served sooner. As wait time grows, even an 80%-need car eventually outranks a freshly arrived 20%-need car. k is a tunable aging constant. On top of this, add:

- **Hard wait-time cap** — no car waits beyond a fixed ceiling regardless of score, giving a worst-case guarantee.
- **Periodic re-ranking window** — recompute priorities every few minutes instead of on every micro-event, avoiding pathological reshuffling when the queue is long and volatile.

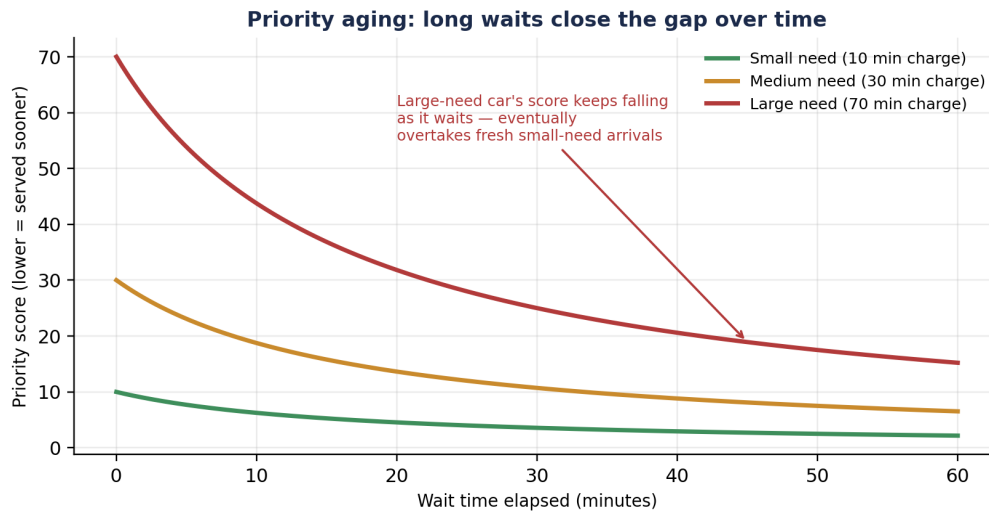


Figure 4. Priority aging in action — a large-need car's score keeps dropping the longer it waits, eventually overtaking newly arrived small-need cars and preventing starvation.

4.4 Practical guards

- Changeover buffer (2–5 min) added to every bay's predicted free time.
- SOC input via OCPP/ISO 15118 where hardware allows; app-based self-report with plausibility checks as an MVP fallback.
- Re-evaluate the whole schedule on every state change (car arrives, car leaves, bay frees) rather than computing it once.

5. Allocation Strategies Compared

Strategy	Basis	Key flaw
First-Come-First-Served	Arrival order only	No regard for charge needed; bays occupied far longer than necessary
Pure SRPT (shortest need first)	Charge time needed	Starves large-need cars under sustained queue load
SOC-aware + aging (proposed)	Charge time needed, weighted by wait time, capacity-matched	Requires accurate telemetry and tuning of aging constant k
Auction / mechanism design	Reported user preferences, market clearing	Complex to implement; assumes truthful reporting

6. Pseudocode Sketch

```

on_event(bay_freed OR car_joins_queue):
    for each waiting car c:
        for each bay b (occupied or free):
            eff_power = min(bay[b].rated_kw, c.max_accept_kw)
            est_free_time[b] = predicted_free_time(b) # curve-based, + changeover buffer
            eta[c][b] = est_free_time[b] + time_to_target(c.kwh_needed, eff_power, charge_
curve)

```

```

for each waiting car c:
    priority[c] = min_b(eta[c][b]) / (1 + k * c.wait_time_elapsed)

assignment = min_cost_bipartite_matching(cars, bays, cost = eta, tie_break = priority)
enforce: no car exceeds MAX_WAIT
apply(assignment)
notify assigned drivers with changeover window

```

7. Feasibility as a Build/Publication Target

This is realistic to prototype in two tracks: (1) a **software simulation** — queueing model in Python with synthetic car arrivals, SOC curves, and bay ratings, benchmarking FCFS vs. SRPT vs. the aging-weighted matcher on total wait time and fairness (e.g. Jain's fairness index); (2) a small **hardware demo** — a handful of simulated "bays" (relay + current sense per channel) driven by an ESP32, with a mock SOC feed standing in for OCPP/ISO 15118 telemetry. Framed as "SOC-aware dynamic bay allocation for multi-bay EV charging stations," this fits well as an Anna University journal piece or IEEE student conference submission, and the simulation alone is enough to generate the comparative results such a paper would need.

8. Reference Sources

- [1] Development of Charging/Discharging Scheduling Algorithm for Economical and Energy-Efficient Operation of Multi-EV Charging Station — MDPI Applied Sciences, 2022.
- [2] Online EV Scheduling Algorithms for Adaptive Charging (FOCS) — UMass CS group.
- [3] Microgrid Revenue Maximization by Charging Scheduling of EVs in Multiple Parking Stations — arXiv:1606.01740.
- [4] Electric Vehicles Charging Station Allocation Based on Load Profile Forecasting and Dijkstra's Algorithm — Scientific Reports, 2025.
- [5] Learning to Operate an Electric Vehicle Charging Station Considering Vehicle-Grid Integration — arXiv:2111.01294.
- [6] Mechanism Design for Efficient Allocation of Electric Vehicles to Charging Stations — arXiv:2007.09715.
- [7] Event-Driven Real-Time Multi-Objective Charging Schedule Optimization for EV Fleets — arXiv:2409.12439.
- [8] An AI-Based Scheduling Algorithm for EV Charging — IJNRD, 2024.
- [9] US20140125485A1 / WO2014074425A2 — Management of Electric Vehicle Charging Station Queues (Google Patents).
- [10] US9142978B2 / US20140125279A1 — Queue Prioritization for Electric Vehicle Charging Stations (Google Patents).
- [11] Dynamic Routing for the Electric Vehicle Shortest Path Problem with Charging Station Occupancy Information — arXiv:2305.11773 / ScienceDirect.
- [12] Proposal for a Queueing System for Public EV Chargers — Bryce Energy Services, 2025.
- [13] On-Demand Valet Charging for Electric Vehicles: Economic Equilibrium, Infrastructure Planning and Regulatory Incentives — arXiv:2106.08162.